

Python Clinic Session 1 for EE2213

Yunze Leng (GA): yleng@u.nus.edu

Yutong Du (TA): yutongdu@nus.edu.sg

©National University of Singapore



August 12, 2026

Contents

1	Anaconda and Conda	1
1.1	What is Anaconda?	1
1.2	Installation	1
1.3	Conda Command Line Interface	1
1.3.1	Why Use Conda Environments?	1
1.3.2	Conda Hands-On	2
1.3.3	Conda Cheat Sheet	2
2	Jupyter Notebook	3
2.1	Advantages of Jupyter Notebook	3
2.2	Understanding the Jupyter Kernel	3
2.3	When to Use Jupyter vs. Traditional IDEs	3
2.4	Installation	3
2.5	Launching Jupyter Notebook	4
2.6	Alternative: Google Colab	4
2.7	Jupyter Cheat Sheet	5
3	IDEs	5
3.1	PyCharm	5
3.2	VSCode	5
4	Assignment Guidance	5
4.1	Test Code before Submission	5
4.2	Common Mistakes	7
4.2.1	File, Function, and Naming Conventions	7
4.2.2	Function and Code Structure	8
4.2.3	Algorithm, Syntax, and Numerical Errors	8
5	Installing Python Alone without Anaconda	9
5.1	Install Python on Mac/Windows	9
5.2	Quick Start	9
5.3	Package Management with pip	10

1 Anaconda and Conda

1.1 What is Anaconda?

Anaconda is the recommended distribution platform for Python that aims to simplify package management and deployment. When you install Anaconda, the Python interpreter is included, so there's no need to install Python separately from the official website. The `pip` package manager comes pre-configured and ready to use. Additionally, Anaconda includes its own powerful `conda` package manager, which provides more advanced environment and package management capabilities than `pip` alone.

You can refer to this YouTube video for more details:

https://www.youtube.com/watch?v=iaNEQglCF-I&ab_channel=SusanB.

1.2 Installation

Download Anaconda from <https://www.anaconda.com/download>.

The detailed installation tutorials can be found in the following YouTube videos:

For MacOS: https://www.youtube.com/watch?v=V4riykgUS94&ab_channel=SusanB.

For Windows: https://www.youtube.com/watch?v=4DQGBQMvwZo&ab_channel=SusanB.

1.3 Conda Command Line Interface

Conda is a powerful package and environment management system that comes with Anaconda. It allows you to create isolated environments for different projects, each with their own packages and Python versions.

1.3.1 Why Use Conda Environments?

Isolated environments are crucial because:

- Different tasks may require different packages or Python versions
- Avoid conflicts between package dependencies
- Keep projects organized and reproducible
- Easy to share environment specifications with others

```
1 pip freeze > requirements.txt
2 # then you can reproduce packages by:
3 pip install -r requirements.txt
4
```

```
5 # or
6 conda env export > environment.yml
7 # then you can create env from yaml file
8 conda env create -f environment.yml
```

1.3.2 Conda Hands-On

Make sure to activate the specific conda environment before installing packages.

```
1 # Create an environment
2 conda create --name EE2213
3
4 # Create an environment with a specific Python version
5 conda create --name EE2213 python=3.11
6
7 # List all environments
8 conda info --envs
9
10 # Activate an environment
11 conda activate EE2213
12 # Deactivate an environment
13 conda deactivate
14
15 # List installed packages in the current environment
16 conda list
17
18 # Install packages from conda
19 conda install pulp
20 # You can also install packages using pip within a conda environment
21 pip install cvxpy
22
23 # Remove packages
24 conda remove numpy
25 # Or
26 conda uninstall numpy
27
28 # Delete entire environment
29 conda remove --name EE2213 --all
```

1.3.3 Conda Cheat Sheet

Detailed commands of conda can be found at:

<https://docs.conda.io/projects/conda/en/latest/user-guide/cheatsheet.html>

2 Jupyter Notebook

2.1 Advantages of Jupyter Notebook

Jupyter Notebook is an interactive computing environment that offers several advantages for data science, research, and educational purposes:

- **Interactive Development:** Execute code in small chunks (cells) and see results immediately
- **Rich Output:** Display plots, images, HTML directly in the notebook
- **Documentation Integration:** Combine code, text (markdown), equations, and visualizations in one document
- **Exploratory Analysis:** Perfect for data exploration and experimental coding

2.2 Understanding the Jupyter Kernel

An important concept in Jupyter is the **kernel**, which is the computational engine that executes your code. If you don't restart the kernel, all previously executed functions, variables, and imported modules remain available in memory and can be directly used in new cells without redefining them. For example, if you define a variable `x = 10` in cell 1, you can use `x` in cell 5 without redefining it.

2.3 When to Use Jupyter vs. Traditional IDEs

Jupyter Notebook is ideal for experimental and interactive work. It allows you to easily obtain intermediate results and visualizations as you write code, making it perfect for data analysis, learning programming concepts, and testing new ideas. The cell-by-cell execution lets you see outputs immediately and modify your approach as needed.

Traditional IDEs are better suited for software development and building complete applications. They provide better tools for writing, organizing, and debugging larger codebases. Use `.py` files when you need to create programs that will run from start to finish, or when working on projects that require proper version control and collaboration.

2.4 Installation

To install Jupyter Notebook using conda, activate your desired environment and run:

```
1 conda install jupyter
```

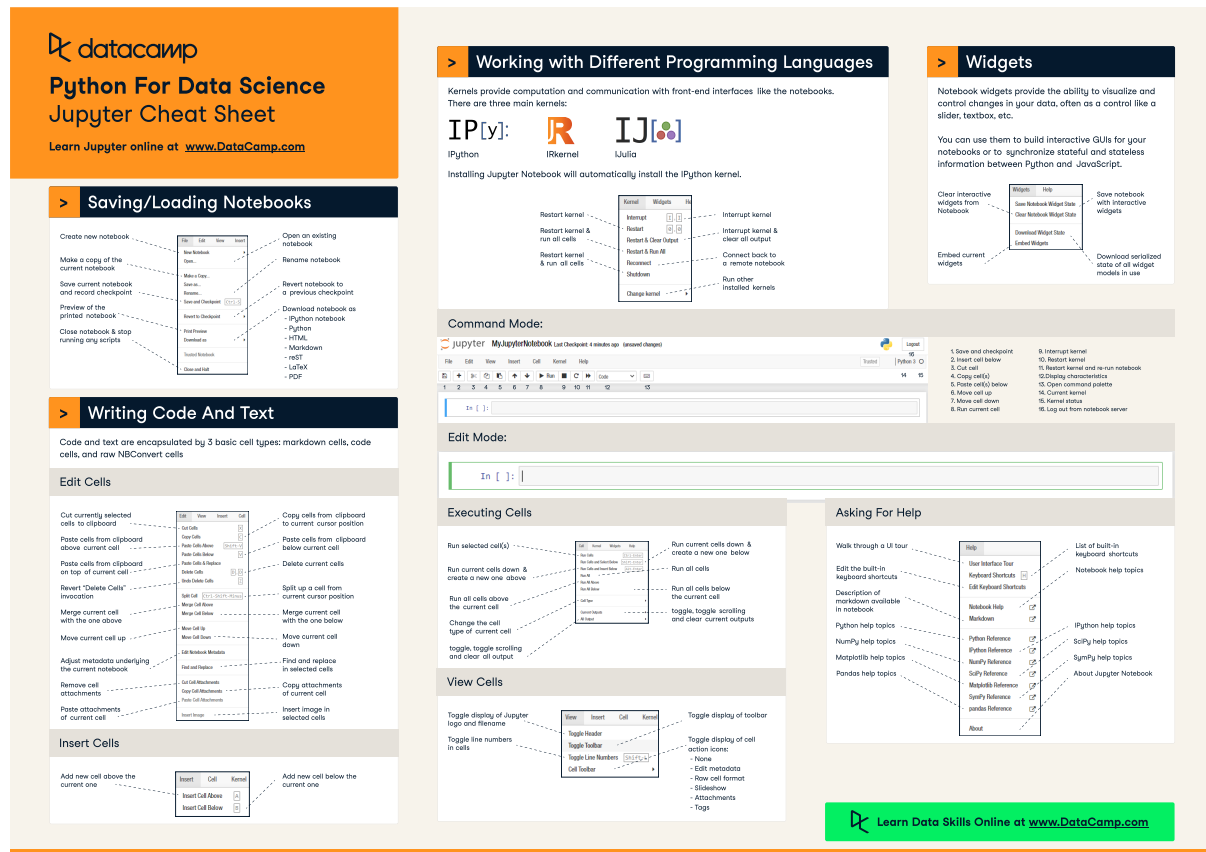


Figure 1: Jupyter cheat sheet.

2.5 Launching Jupyter Notebook

After installation, launch Jupyter Notebook by running:

```
1 jupyter notebook
```

This command will (1) start the Jupyter server; (2) open your default web browser; (3) display the Jupyter dashboard showing your file system.

2.6 Alternative: Google Colab

For those who have issues with local Jupyter installation or prefer cloud-based solutions, you can use Google Colab instead: <https://colab.google.com/>

Note: For EE2211/EE2213 students, Colab requires Internet connection, so it is **NOT** suitable for your ExamSoft-based quizzes!

2.7 Jupyter Cheat Sheet

See Figure 1.

3 IDEs

Integrated Development Environments (IDEs) provide comprehensive tools for Python development, including syntax highlighting, debugging, version control integration, and project management. Here are two popular choices for Python development.

3.1 PyCharm

Download PyCharm from here: <https://www.jetbrains.com/pycharm/download>

Instructions for setting up a Python environment in PyCharm: <https://www.jetbrains.com/help/pycharm/configuring-python-interpreter.html>

3.2 VSCode

Download VSCode from here: <https://code.visualstudio.com/download>

After installation, you'll need to install the Python extension and configure your Python environment. Detailed instructions for setting up a Python environment can be found here: <https://code.visualstudio.com/docs/python/environments>

4 Assignment Guidance

4.1 Test Code before Submission

Assuming that the question asks us to compute the Euclidean distance and Manhattan distance between two arbitrary vectors $\mathbf{x}$ and $\mathbf{y}$. $\mathbf{x} \in \mathbb{R}^n$ and $\mathbf{y} \in \mathbb{R}^n$. Suppose your assignment answer file that is to be submitted is as follows:

```
1 import numpy as np
2
3 # Please replace "MatricNumber" with your actual matric number here and in the
   ↪ filename
4 def A1_MatricNumber(x, y):
5     """
6     Input type
7     :x type: numpy.ndarray
8     :y type: numpy.ndarray
9     Return type
10    :euclidean_dist type: numpy.ndarray
```

```

11     :manhattan_dist type: numpy.ndarray
12     """
13
14     # your code goes here
15     pass
16
17     # return in this order
18     return euclidean_dist, manhattan_dist

```

Please test your code before submitting. You can either test on your original file:

```

1 import numpy as np
2 # Please replace "MatricNumber" with your actual matric number here and in the
   ↪ filename
3 def A1_A1234567R(x, y):
4     """
5     Input type
6     :x type: numpy.ndarray
7     :y type: numpy.ndarray
8     Return type
9     :euclidean_dist type: numpy.ndarray
10    :manhattan_dist type: numpy.ndarray
11    """
12
13    # your code goes here
14    euclidean_dist = # compute Euclidean distance
15    manhattan_dist = # compute Manhattan distance
16    # return in this order
17    return euclidean_dist, manhattan_dist
18
19 # Test with sample data
20 x = np.array([1, 2, 3])
21 y = np.array([4, 5, 6])
22 euclidean_dist, manhattan_dist = A1_A1234567R(x, y)
23 print(f"Euclidean distance: {euclidean_dist}")
24 print(f"Manhattan distance: {manhattan_dist}")

```

You can debug this way, but please remember to **delete** the testing lines (i.e., the lines after the function definition). The final script for submission should only include the function and the necessary imports.

Or use the provided "test.py" file:

```

1 import numpy as np
2 import A1_A1234567R as grading # Make sure this filename matches the student's
   ↪ submission

```



```

3
4 # Define test inputs
5 x = np.array([1, 2, 3])
6 y = np.array([4, 0, 3])
7
8 # Call the student function
9 try:
10     euclidean_dist, manhattan_dist = grading.A1_A1234567R(x, y)
11
12     print("Euclidean Distance:", euclidean_dist[0])    # Expected: sqrt((1-4)^2 + (2-0)
    ↪ ^2 + (3-3)^2) = sqrt(9 + 4 + 0) = sqrt(13)
13     print("Manhattan Distance:", manhattan_dist[0])    # Expected: |1-4| + |2-0| +
    ↪ |3-3| = 3 + 2 + 0 = 5
14
15 except Exception as e:
16     print("Error during grading:", str(e))

```

To do this, make sure you place the `test.py` file in the **exact same directory** as your solution file (e.g., `A1_A1234567R.py`). When the testing script tries to run `import A1_A1234567R as grading`, Python looks for your file in the current folder. If they are not together, you will encounter a `ModuleNotFoundError`.

Then run `test.py` either from the Python console or terminal, the output should be like this:

```

1 # Euclidean Distance: 3.605551275463989
2 # Manhattan Distance: 5

```

If you encounter an error like this during testing:

```

1 Error during grading: name 'euclidedn_dist' is not defined

```

Please check for typos in variable names (in this case, “euclidedn_dist” should be “euclidean_dist”).

4.2 Common Mistakes

This section highlights common errors observed in previous years. Please read these points carefully to ensure your submission is correct and to avoid unnecessary penalties. Many of these issues can be easily caught by testing your code before submission.

4.2.1 File, Function, and Naming Conventions

The grading script automatically finds and runs your code based on strict naming conventions. Errors in this category can lead to the script failing to grade your work,

resulting in a score of zero.

- **Correct File Name and Type:** Ensure your file is named exactly as specified, for example `A1234567R.py`. Make sure your operating system is configured to show file extensions to avoid submitting files named `A1234567R.py.py`. You must submit the exact file format we require (e.g., `.py` or `.ipynb`) as specified in the assignment details.
- **Correct Function Name:** The function name in your code must exactly match the one provided in the assignment template, including your student ID if required.
- **Case Sensitivity:** Python is case-sensitive. A variable named `Y_train` is different from `y_train`. Use the exact variable names given in the template (if provided) for function arguments, local variables, and return values.

4.2.2 Function and Code Structure

A function should be a self-contained and clean block of code that processes inputs to produce outputs according to the assignment’s specifications.

- **Imports at the Top:** All `import` statements (e.g., `import numpy as np`, `from sklearn...`) must be placed at the very top of your `.py` file. Do not place them inside your function definitions.
- **Understanding Local Imports:** When you use “from filename import function_name”, the `filename` should not include the `.py` extension. This works when the file you are importing from is in the same folder as the script you are running.
- **Use, Don’t Redefine Inputs:** Your function should directly use the input arguments (e.g., `x`, `y`) if provided. Do not overwrite or redefine them inside your function.
- **Clean Code:** Before submitting, remove all debugging lines, such as intermediate `print()` statements or unused code snippets.

4.2.3 Algorithm, Syntax, and Numerical Errors

These errors relate to the core logic, syntax, and numerical stability of your implementation.

- **Basic Syntax Errors:** Double-check your code for fundamental Python errors like incorrect indentation, missing colons (`:`), or mismatched parentheses (`()`). These will prevent your program from running at all.

- **Shape Mismatches and Input Validation:** Always be aware of your NumPy array dimensions (`.shape`). Operations like matrix multiplication, `reshape`, and functions like `train_test_split` are highly sensitive to array shapes.
- **Undefined Variables:** Ensure every variable is assigned a value before it is used in a calculation or returned.
- **Correct Operators/Methods:** Use the correct methods for a given object type. For example, you cannot use the `.append()` method on a NumPy array; you would typically use a function like `np.append()`.
- **Numerical Stability:** Watch out for numerical issues like an `OverflowError`, which can occur if numbers in your calculations become too large to be represented. This often points to an algorithmic flaw, such as a learning rate that is too high in gradient descent.

5 Installing Python Alone without Anaconda

5.1 Install Python on Mac/Windows

While Anaconda provides a comprehensive Python distribution with many pre-installed packages, you may sometimes prefer to install Python directly from the official source. This approach gives you more control over your Python environment and results in a lighter installation.

- Visit <https://www.python.org/downloads/>
- Download the latest Python version for macOS/Windows
- Run the installer and follow the installation wizard
- Python will be installed with `pip` included

5.2 Quick Start

Once installed, you can start using Python immediately. Here we demonstrate using macOS terminal, but the commands are similar on Windows Command Prompt or PowerShell.

```
1 # Check Python version
2 python3 --version
3
4 # Create a Python file
```

```
5 echo 'print("Hello world!")' > hello_world.py
6
7 # View the file content
8 cat hello_world.py
9
10 # Run the Python program
11 python3 ./hello_world.py
```

5.3 Package Management with pip

You can install additional packages as needed using `pip`.

```
1 # Check pip version
2 pip3 --version
3
4 # Installing packages
5 pip3 install <package_name>
6 # For example
7 pip3 install requests
8
9 # Show installed packages
10 pip3 list
11
12 # Show package information
13 pip3 show <package_name>
14
15 # Uninstall a package
16 pip3 uninstall <package_name>
17
18 # Upgrade a package
19 pip3 install --upgrade <package_name>
```